

OSSP Practical session-11

Task-1:

Develop a multithreaded counter application using `pthread_create()` and `pthread_join()`. Multiple threads update a shared variable concurrently to demonstrate a race condition.

Source code:

```
GNU nano 7.2 thread_counter.c *
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define INCREMENTS 100000

int counter = 0;

void *increment_counter(void *arg)
{
    for (int i = 0; i < INCREMENTS; i++)
    {
        counter++;
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    printf("MULTITHREADED COUNTER - RACE CONDITION\n");
    printf("-----\n");

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i], NULL,
                      increment_counter, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter value: %d\n",
```

```
int main()
{
    pthread_t threads[NUM_THREADS];

    printf("MULTITHREADED COUNTER - RACE CONDITION\n");
    printf("-----\n");

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i], NULL,
                      increment_counter, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter value: %d\n",
          NUM_THREADS * INCREMENTS);

    printf("Actual counter value:  %d\n", counter);

    return 0;
}
```

Output:

```
PS C:\Users\SAIRAM0806> wsl
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ mkdir thread_lab
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ cd thread_lab
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806/thread_lab$ nano thread_counter.c
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806/thread_lab$ gcc thread_counter.c -o thread_counter -pthread
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806/thread_lab$ ls
thread_counter  thread_counter.c
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806/thread_lab$ ./thread_counter
MULTITHREADED COUNTER - RACE CONDITION
-----
Expected counter value: 400000
Actual counter value:   400000
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806/thread_lab$ ./thread_counter
MULTITHREADED COUNTER - RACE CONDITION
-----
Expected counter value: 400000
Actual counter value:   400000
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806/thread_lab$
```

Observation:

The multithreaded counter program was executed using POSIX threads. Multiple threads updated the same shared counter simultaneously, resulting in a race condition. The final counter value may be less than the expected value due to concurrent access.

Task-2: Modify the previous program using `pthread_mutex_lock()` and `pthread_mutex_unlock()` and compare the output with the race-condition version.

```
GNU nano 7.2                                counter_mutex.c *
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define COUNT 100000

int counter = 0;
pthread_mutex_t lock;

void *increment(void *arg)
{
    for (int i = 0; i < COUNT; i++)
    {
        pthread_mutex_lock(&lock);

        counter++;

        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    pthread_mutex_init(&lock, NULL);

    printf("Initial counter value: %d\n", counter);

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i], NULL, increment, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Final counter value: %d\n", counter);

    pthread_mutex_destroy(&lock);

    return 0;
}
```

Output:

```
PS C:\Users\SAIRAM0806> wsl
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ mkdir thread_lab
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ cd thread_lab
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806/thread_lab$ nano thread_counter.c
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806/thread_lab$ gcc thread_counter.c -o thread_counter -pthread
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806/thread_lab$ ls
thread_counter  thread_counter.c
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806/thread_lab$ ./thread_counter
MULTITHREADED COUNTER - RACE CONDITION
-----
Expected counter value: 400000
Actual counter value:   400000
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806/thread_lab$ ./thread_counter
MULTITHREADED COUNTER - RACE CONDITION
-----
Expected counter value: 400000
Actual counter value:   400000
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806/thread_lab$
```

Observation:

The counter program was modified using mutex locks with `pthread_mutex_lock()` and `pthread_mutex_unlock()`. The mutex provided synchronization between threads and prevented the race condition, giving the correct final counter value of 400000.